

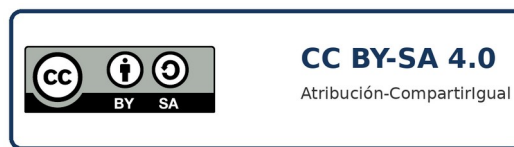
**MATERIAL DOCENTE EN ABIERTO
DE LA UNIVERSIDAD REY JUAN CARLOS**

Apuntes de Big Data Processing II

Máster Universitario en Análisis de Datos Deportivos

Curso 2026-2027

Alberto Fernández Isabel · Natalia Madrueño Sierro · Rubén Rodríguez Fernández



Depósito institucional BURJC Digital

<https://burjcdigital.urjc.es>

Información de publicación

Material docente en abierto de la Universidad Rey Juan Carlos.

© 2026 Alberto Fernández Isabel, Natalia Madrueño Sierro y Rubén Rodríguez Fernández.
Algunos derechos reservados.

Este documento se distribuye bajo la licencia [Creative Commons Atribución-CompartirIgual 4.0 Internacional](https://creativecommons.org/licenses/by-sa/4.0/)

La licencia se aplica al texto, los ejemplos, las tablas y los esquemas originales. Los nombres, logotipos, marcas, citas y materiales de terceros conservan su régimen jurídico propio y quedan excluidos de la licencia, salvo indicación expresa.

Esta edición no incorpora fotografías ni ilustraciones de terceros. Los fragmentos de código se presentan con finalidad docente y forman parte del contenido original de los apuntes.

Asignatura: Big Data Processing II. Titulación: Máster Universitario en Análisis de Datos Deportivos. Lugar previsto de depósito: BURJC Digital. Fecha de esta edición: 2026.

Índice

Nivel	Contenido
Tema	1. Procesamiento distribuido de Big Data
Tema	2. Kafka y Spark Structured Streaming
Tema	3. Optimización y observabilidad
Tema	4. IA generativa, RAG y agentes
Tema	5. Proyecto integrado
Tema	6. Tendencias y gobernanza
Tema	7. Guía de prácticas
Tema	8. Autoevaluación
Tema	9. Glosario
Tema	10. Bibliografía y documentación

Presentación

Big Data Processing II estudia cómo diseñar sistemas que reciben, procesan, almacenan y explotan datos a gran velocidad. La asignatura combina procesamiento distribuido, Apache Kafka, Spark Structured Streaming, optimización, inteligencia artificial generativa, recuperación aumentada y sistemas basados en agentes.

El contexto conductor es la analítica deportiva. Un partido o entrenamiento produce eventos procedentes de marcadores, tracking, dispositivos vestibles, sensores, aplicaciones y documentos. El reto no es solo acumularlos: hay que preservar su significado temporal, controlar duplicados y retrasos, calcular métricas reproducibles y transformar los resultados en información útil.

Los apuntes siguen seis temas:

1. procesamiento distribuido y arquitecturas Big Data;
2. procesamiento en tiempo real con Kafka y Spark;
3. optimización, escalabilidad y observabilidad;
4. modelos generativos, RAG y agentes;
5. integración de un sistema completo;
6. tendencias, gobernanza y uso responsable.

Cada tema conecta conceptos, decisiones de diseño, ejemplos deportivos y prácticas. El objetivo no es memorizar productos, sino justificar una arquitectura a partir de requisitos verificables.

1. Procesamiento distribuido de Big Data

1.1. Motivación

Un sistema centralizado ejecuta todo en una máquina. Es sencillo, pero queda limitado por sus recursos y constituye un punto único de fallo. Un sistema distribuido reparte datos y cómputo entre varios nodos.

La distribución responde a cuatro necesidades:

- **volumen:** los datos no caben o no se procesan a tiempo en un nodo;
- **velocidad:** el paralelismo reduce el tiempo de respuesta;
- **disponibilidad:** el servicio continúa si falla una máquina;
- **elasticidad:** la capacidad se adapta a la carga.

Distribuir no elimina complejidad: aparecen redes variables, relojes no perfectamente sincronizados, fallos parciales, particiones descompensadas y operaciones repetidas.

1.2. Escalado vertical y horizontal

El escalado vertical aumenta CPU, memoria o almacenamiento de una máquina. Es sencillo, pero tiene límites físicos y económicos. El horizontal añade nodos y reparte la carga. Aporta elasticidad y tolerancia a fallos, pero exige particionado, coordinación y observabilidad.

Una prueba deportiva puede funcionar localmente. Una competición con encuentros simultáneos, miles de deportistas y sensores múltiples requiere distribuir ingesta y procesamiento.

1.3. Batch, microbatch y streaming

En **batch**, los datos se acumulan y procesan como conjunto finito. Es adecuado para estadísticas históricas, entrenamiento de modelos o informes diarios.

En **streaming**, los eventos se procesan continuamente. Es útil cuando el valor disminuye con el tiempo: alertas, incidencias o analítica en directo.

El **microbatch** agrupa eventos durante intervalos breves. Spark Structured Streaming utiliza normalmente este modelo. “Tiempo real” no significa siempre milisegundos: una retransmisión puede aceptar segundos; una alerta de seguridad quizá necesite cientos de milisegundos.

1.4. Semántica temporal

Un evento puede tener:

- **event time**: cuándo ocurrió;
- **ingestion time**: cuándo entró;
- **processing time**: cuándo se procesó.

Una carrera registrada a las 18:02 puede llegar a las 18:05. Agruparla por processing time la atribuiría al intervalo equivocado.

Ventanas habituales:

- *tumbling*: intervalos consecutivos sin solapamiento;
- *sliding/hopping*: ventanas solapadas que avanzan con un salto;
- *session*: agrupan actividad separada por inactividad.

El **watermark** expresa cuánto retraso se acepta. Permite liberar estado, pero obliga a decidir qué hacer con eventos posteriores: descartarlos, almacenarlos aparte o corregir resultados publicados.

1.5. Particionado

El particionado asigna cada evento a una unidad de almacenamiento o procesamiento. Una clave adecuada mantiene juntos datos que deben ordenarse o agregarse.

Ejemplos:

- `match_id` mantiene eventos de un partido;
- `player_id` facilita métricas por deportista;
- una clave compuesta puede equilibrar partidos muy activos.

Una mala clave produce **skew**: una partición recibe la mayor parte del tráfico. La clave se elige por semántica, distribución y patrón de consulta.

1.6. Replicación y CAP

Replicar mantiene copias en varios nodos. Mejora disponibilidad y durabilidad, pero exige coordinación.

CAP indica que, durante una partición de red, hay que priorizar consistencia o disponibilidad. No obliga a elegir permanentemente una propiedad: describe el compromiso cuando falla la comunicación.

El marcador oficial puede necesitar más consistencia que un panel exploratorio.

1.7. Lambda y Kappa

Lambda combina una capa batch, que recalcula resultados completos, y una capa de velocidad, que produce resultados recientes. Es robusta, pero duplica lógica.

Kappa trata todo como flujo reproducible. Los cambios se aplican reprocesando el registro de eventos. Simplifica el modelo, pero requiere retención y procesadores capaces de reconstruir estado.

1.8. Semánticas de entrega e idempotencia

- **At most once:** puede perderse un mensaje, pero no repetirse.
- **At least once:** no debería perderse, pero puede duplicarse.
- **Exactly once:** el efecto observable se produce una vez bajo condiciones concretas.

Exactly once debe analizarse extremo a extremo. Una API externa no idempotente puede duplicar efectos aunque el motor gestione bien sus checkpoints.

Una operación idempotente produce el mismo resultado al repetirse. Identificadores de evento, claves deterministas y operaciones *upsert* ayudan a controlar duplicados.

1.9. Decisiones mínimas

Antes de implementar deben documentarse:

7. fuentes y tasa esperada;
8. latencia objetivo;
9. clave de particionado;
10. política de duplicados y retrasos;
11. estado mantenido;
12. recuperación;
13. destinos;
14. métricas de éxito.

2. Kafka y Spark Structured Streaming

2.1. Kafka como registro distribuido

Kafka organiza eventos en **topics**, divididos en **particiones**. Dentro de cada partición se conserva orden. Cada mensaje recibe un **offset**.

Componentes:

- productor;
- broker;
- consumidor;
- consumer group;
- controlador;
- réplicas.

Kafka no es solo una cola: conserva eventos, permite releerlos y admite consumidores independientes.

2.2. Diseño de topics

Una estructura posible:

- sports.raw.events;
- sports.validated.events;
- sports.metrics;
- sports.alerts;
- sports.dead-letter.

Para cada topic deben definirse esquema, clave, particiones, replicación, retención, compactación y responsables.

Crear un topic por deportista o partido suele ser inmanejable. Es preferible agrupar y usar claves.

2.3. Productores

El productor serializa, selecciona partición y espera confirmación. Deben configurarse:

- acknowledgements;
- reintentos;
- batching;
- compresión;
- idempotencia;
- timeout;
- validación de esquema.

Evento conceptual:

```
{
  "event_id": "evt-1042",
  "timestamp": "2026-10-01T18:24:12Z",
  "match_id": "match-27",
  "team": "home",
  "player": "p-8",
  "event_type": "shot",
  "zone": "box",
  "minute": 63,
  "value": 0.18
}
```

Usar `match_id` como clave conserva el orden relativo del encuentro dentro de una partición.

2.4. Consumidores

Cada partición se asigna como máximo a un consumidor del mismo grupo. Más consumidores que particiones no aumentan paralelismo.

Debe decidirse:

- cuándo confirmar offsets;
- qué ocurre si falla el trabajo;
- cómo responder a rebalances;
- qué hacer con mensajes inválidos;
- cuánto tarda un lote.

Confirmar antes del procesamiento puede producir pérdida lógica. Confirmar después exige tolerar repeticiones.

2.5. Replicación y KRaft

Cada partición tiene líder y réplicas. Si falla el líder, una réplica sincronizada puede asumirlo.

Kafka moderno utiliza KRaft y un quorum Raft para metadatos, sustituyendo la dependencia histórica de ZooKeeper. La disponibilidad sigue dependiendo de réplicas actualizadas y configuración coherente.

2.6. Esquemas

JSON es cómodo, pero producción necesita contratos. Avro, Protobuf o JSON Schema permiten validar y evolucionar.

Añadir un campo opcional suele ser compatible; eliminar o cambiar tipos puede romper consumidores. El esquema debe tratarse como API: versionado, compatibilidad, validación, documentación y pruebas.

2.7. Structured Streaming

Structured Streaming representa un flujo como una tabla que crece. El flujo habitual es:

15. leer Kafka;
16. convertir clave y valor;
17. aplicar esquema;
18. validar;

19. transformar;
20. agregar;
21. escribir.

Ejemplo abreviado:

```
raw = (
    spark.readStream
        .format("kafka")
        .option("kafka.bootstrap.servers", brokers)
        .option("subscribe", "sports.raw.events")
        .load()
)

events = (
    raw.selectExpr("CAST(value AS STRING) AS json")
        .select(from_json(col("json"), schema).alias("e"))
        .select("e.*")
)
```

El esquema explícito evita inferencias inconsistentes.

2.8. Ventanas y watermarks

```
metrics = (
    events
        .withWatermark("timestamp", "10 minutes")
        .groupBy(
            window(col("timestamp"), "5 minutes", "1 minute"),
            col("match_id"),
            col("team")
        )
        .agg(count("*").alias("events"))
)
```

La ventana dura cinco minutos y avanza cada minuto. El watermark permite cerrar estado según la tolerancia definida.

2.9. Modos de salida

- **append:** solo filas finales;
- **update:** filas modificadas;
- **complete:** toda la tabla agregada.

El modo debe ser compatible con consulta y destino. Complete puede resultar costoso con estados grandes.

2.10. Checkpoints

Los checkpoints conservan offsets y estado para reanudar. Deben almacenarse en un sistema duradero y cada consulta debe usar una ubicación propia.

Cambiar radicalmente una consulta reutilizando su checkpoint puede ser incompatible. Consulta, salida y checkpoint deben versionarse.

2.11. Datos tardíos e inválidos

Un pipeline debe separar eventos válidos, tardíos, inválidos y fallos transitorios.

Una dead-letter queue conserva mensajes no procesables con motivo, momento y versión del esquema. Necesita métricas y procedimiento de reprocesado.

2.12. Ejercicio integrado

Construya un productor y una consulta Spark que:

- valide esquema;
- calcule tiros y goles;
- agrupe por ventanas;
- use watermark;
- persista resultados;
- documente duplicados, eventos tardíos y recuperación.

3. Optimización y observabilidad

3.1. Medir primero

Defina:

- tasa de entrada y procesada;
- latencia y percentiles p90/p95/p99;
- CPU y memoria;
- tamaño del estado;
- lag;
- duración de microbatches;
- errores y reintentos.

El promedio oculta episodios críticos. Un p95 de ocho segundos significa que el 5 % supera ese valor.

3.2. Paralelismo

Está limitado por particiones, ejecutores y distribución de claves. Pocas particiones limitan; demasiadas incrementan metadatos, ficheros pequeños y coordinación. Debe decidirse mediante pruebas.

3.3. Shuffle

Un shuffle redistribuye datos para agrupar, unir u ordenar. Consume red, serialización, disco y memoria.

Para reducirlo:

- filtrar pronto;
- seleccionar columnas necesarias;
- agregar localmente;
- evitar reparticionados;
- equilibrar claves;
- usar broadcast join solo con tablas realmente pequeñas.

3.4. Skew

Síntomas:

- pocas tareas mucho más lentas;
- memoria desigual;
- colas crecientes con recursos libres.

Soluciones:

- salting;
- agregación en dos fases;
- separar claves calientes;
- aumentar particiones;

- rediseñar clave.

3.5. Estado

Ventanas y deduplicación mantienen estado. Si no se acota, crece indefinidamente. Se controla con watermarks, expiración, ventanas limitadas, cardinalidad y monitorización.

3.6. Formatos y compresión

Kafka usa compresión para reducir red y almacenamiento. Parquet facilita lectura columnar y filtrado. CSV es interoperable, pero pierde tipos y ocupa más.

La elección depende de lectura, tamaño, tipos, evolución y coste.

3.7. Caché

Persistir ayuda si un DataFrame se reutiliza y recomputarlo es caro. Cachear todo consume memoria y puede empeorar el sistema. Debe medirse y liberarse.

3.8. Backpressure

Si la entrada supera la capacidad, crecen lag y latencia. Opciones:

- escalar;
- aumentar particiones;
- reducir coste;
- limitar lectura;
- degradar funciones no esenciales;
- priorizar.

3.9. Observabilidad

Tres pilares:

- métricas;
- logs;
- trazas.

Conviene propagar event_id, match_id, versión de esquema y versión de procesamiento.

Kafka: producción, consumo, lag, réplicas, errores.

Spark: duración de lote, filas, memoria, spill, estado y tareas fallidas.

3.10. Experimentos reproducibles

Toda optimización debe registrar hipótesis, línea base, carga, cambio aislado, métricas, efectos secundarios y conclusión. Mayor velocidad con pérdida de corrección no es mejora global.

4. IA generativa, RAG y agentes

4.1. Papel del modelo

Un modelo puede resumir métricas o adaptar explicaciones. No debe sustituir cálculos deterministas.

Deben separarse:

- hechos calculados;
- contexto recuperado;
- instrucciones;
- salida generada;
- evidencias;
- validaciones.

4.2. Prompt engineering

Un prompt robusto incluye contexto, tarea, datos, restricciones, formato y conducta ante información insuficiente.

Ejemplo:

```
Genera un resumen para el cuerpo técnico.
Usa únicamente métricas y evidencias proporcionadas.
Distingue hechos de contexto documental.
No inventes lesiones, causas ni intenciones.
Devuelve resumen, tres hallazgos, limitaciones y evidencias.
```

Few-shot añade ejemplos. Deben ser representativos y coherentes.

4.3. Salidas estructuradas

```
{
  "match_id": "match-27",
  "summary": "...",
  "findings": [
    {"claim": "...", "evidence": ["metric:shots_home"]}
  ],
  "limitations": ["..."]
}
```

Debe validarse sintaxis, campos, tipos, límites y consistencia.

4.4. Post-training

Incluye ajuste supervisado, ajuste eficiente, preferencias, alineamiento y destilación.

No siempre es necesario. Si falta conocimiento actualizado, RAG suele ser mejor. Si se necesita comportamiento repetido o formato estable, el ajuste puede aportar valor.

4.5. RAG

Etapas:

22. ingesta;
23. limpieza;

24. fragmentación;
25. embeddings;
26. indexación;
27. consulta;
28. recuperación;
29. prompt;
30. respuesta;
31. evaluación.

RAG no garantiza veracidad. Puede recuperar contenido irrelevante o ser ignorado.

4.6. Fragmentación y metadatos

Fragmentos pequeños pierden contexto; grandes diluyen señales. Deben conservar fuente, fecha, entidad, tipo, permisos, versión y sección.

4.7. Recuperación

Puede ser densa, léxica o híbrida. MMR favorece diversidad; los filtros restringen por metadatos; un reranker reordena candidatos.

En deporte, conviene filtrar equipo, competición y temporada.

4.8. Evaluación

Recuperación:

- Recall@k;
- Precision@k;
- MRR;
- nDCG.

Respuesta:

- fidelidad;
- cobertura;
- corrección;
- utilidad;
- citas;
- afirmaciones no respaldadas.

4.9. Workflows y agentes

Un workflow ejecuta pasos prefijados. Un agente elige acciones según estado y objetivo.

Componentes:

- estado;
- modelo;
- tools;
- transiciones;
- memoria;

- criterios de parada;
- validación;
- supervisión.

Si la secuencia es conocida, un workflow es más controlable.

4.10. Tools

Una tool necesita nombre, descripción, esquema, salida estructurada, errores, permisos mínimos y límites. Separar lectura y escritura reduce riesgos. Acciones externas deben requerir aprobación.

4.11. LangGraph

Un grafo puede coordinar métricas, recuperación, validación, generación, verificación y revisión humana.

El estado debe ser pequeño, tipado y auditable. Cada nodo registra entradas, salidas y transición.

4.12. Límites

Defina máximo de iteraciones, presupuesto, timeout, detección de repetición, condición de éxito y salida segura.

4.13. Trazabilidad

Toda afirmación debe vincularse con una métrica, consulta, documento o decisión humana. La trazabilidad convierte el informe en producto revisable.

5. Proyecto integrado

5.1. Arquitectura

```

Fuentes -> Kafka -> validación -> Spark -> métricas
                                     |
Documentos -> índice RAG -> recuperación
                                     |
Métricas + contexto -> workflow/agente -> informe -> revisión

```

Cada flecha representa un contrato.

5.2. Capas

- **Ingesta:** recibe, identifica y conserva el original.
- **Procesamiento:** valida, deduplica, gestiona tiempo y agrega.
- **Persistencia:** separa crudo, depurado, métricas y productos.
- **Recuperación:** indexa documentos autorizados.
- **Generación:** crea un borrador estructurado.
- **Gobernanza:** registra versiones, costes, errores y permisos.

5.3. Contrato de eventos

```

{
  "event_id": "evt-1042",
  "schema_version": "1.0",
  "event_time": "2026-10-01T18:24:12Z",
  "match_id": "match-27",
  "team_id": "team-a",
  "player_id": "p-8",
  "event_type": "shot",
  "zone": "box",
  "value": 0.18
}

```

Reglas: identificador único, UTC, catálogos versionados, obligatorios y evolución compatible.

5.4. Métricas

Cada métrica debe indicar nombre, fórmula, unidad, ventana, población, datos faltantes, responsable y versión. Un índice sin fórmula reproducible no es auditable.

5.5. Datos heterogéneos

Se combinan eventos, alineaciones, perfiles, históricos, documentos y catálogos. Deben usarse identificadores estables, no nombres ambiguos.

5.6. Text-to-SQL

Flujo seguro:

32. interpretar;
33. consultar esquema permitido;
34. generar SQL;
35. validar;
36. bloquear escritura;

37. estimar coste;
38. ejecutar con límites;
39. devolver resultado y consulta;
40. explicar.

El modelo no accede a credenciales ni ejecuta sin restricciones.

5.7. Informe

Debe distinguir:

- hechos calculados;
- contexto recuperado;
- interpretación;
- limitaciones.

No debe atribuir causalidad cuando solo hay correlación.

5.8. Pruebas

- unitarias;
- integración;
- reproducción;
- fallos de broker;
- mensajes corruptos;
- duplicados;
- retrasos;
- documentos no autorizados;
- timeout;
- respuesta inválida.

5.9. Operación

Documento variables, dependencias, datos de ejemplo, arranque, health checks, logs, secretos, recuperación y parada.

5.10. Entregables

Código, arquitectura, contratos, pruebas, métricas, resultados, informe, evaluación, limitaciones y decisiones justificadas.

6. Tendencias y gobernanza

6.1. Lakehouse

Combina flexibilidad de data lake con garantías de warehouse. Formatos de tabla abiertos permiten transacciones, evolución de esquema, versiones y batch/streaming común. Evalúe interoperabilidad, catálogo, seguridad y bloqueo de proveedor.

6.2. Streaming avanzado

Tendencias: estado, unificación batch-streaming, consultas incrementales, CDC, event sourcing, grafos y edge. Edge reduce latencia y tráfico, pero complica operación.

6.3. Multimodalidad

Texto, vídeo, audio, tracking y sensores pueden combinarse. Exige sincronización, derechos y evaluación. No debe asumirse comprensión correcta de táctica, biomecánica o medicina.

6.4. Agentes especializados

La coordinación de agentes aumenta modularidad y superficie de fallo. Requiere protocolos, permisos, observabilidad, memoria controlada y resolución de conflictos.

6.5. Privacidad

Los datos deportivos pueden ser personales, biométricos o de salud. Principios: finalidad, minimización, base jurídica, transparencia, acceso, conservación, seguridad y derechos.

Eliminar nombres no garantiza anonimato: trayectorias y atributos pueden reidentificar.

6.6. Sesgos

El rendimiento puede variar por deporte, categoría o perfil. Deben documentarse composición, cobertura, subgrupos, métricas desagregadas y limitaciones.

6.7. Explicabilidad

La explicación se adapta:

- ingeniería: consulta, versión y logs;
- analista: métricas y evidencia;
- entrenador: hallazgos y límites;
- deportista: información comprensible;
- jurídico: datos, finalidad y automatización.

6.8. Seguridad

Amenazas: manipulación, acceso indebido, secretos, prompt injection, tool abuse, dependencias comprometidas, denegación y exfiltración.

Medidas: autenticación, cifrado, segmentación, validación, allowlists, escaneo, registros, límites y revisión humana.

6.9. Gobernanza de modelos

Registre modelo, versión, parámetros, prompt, tools, documentos, fecha, evaluación, aprobaciones, costes e incidentes.

6.10. Coste y sostenibilidad

Más nodos o modelos mayores no implican mejor solución. Evalúe utilización, coste por evento y posibilidad de usar reglas o modelos pequeños.

6.11. Lista de control

41. ¿Finalidad definida?
42. ¿Datos necesarios?
43. ¿Métricas reproducibles?
44. ¿Fuentes autorizadas?
45. ¿Afirmaciones con evidencia?
46. ¿Límites y parada?
47. ¿Salida corregible?
48. ¿Supervisión humana?
49. ¿Evaluación por subgrupos?
50. ¿Trazabilidad?

7. Guía de prácticas

Kafka

Diseñar esquema y clave; generar eventos; configurar productor; comprobar particiones; consumir; simular duplicados; medir lag; documentar recuperación.

Spark

Conectar Kafka; definir esquema; validar; añadir event time; watermark; ventanas; modo de salida; persistencia; checkpoint y reinicio.

Prompting

Definir tarea medible; prompt base; identificar fallos; añadir restricciones; comparar zero-shot/few-shot; validar formato; decidir entre RAG y ajuste.

RAG

Preparar corpus autorizado; metadatos; fragmentar; indexar; preguntas; medir recuperación; generar con evidencias; registrar fallos.

Agentes

Definir estado; tools pequeñas; workflow determinista; añadir decisión agéntica solo donde aporte valor; limitar iteraciones; probar errores; registrar trayectoria.

Proyecto final

Contrato, productor, Spark, métricas, persistencia, corpus, recuperación, orquestación, informe, pruebas y memoria.

8. Autoevaluación

51. ¿Qué diferencia existe entre event time y processing time?
52. ¿Por qué una clave puede crear skew?
53. ¿Cuándo usar batch?
54. ¿Qué representa un offset?
55. ¿Por qué más consumidores que particiones no ayudan?
56. ¿Qué función cumple un watermark?
57. ¿Qué diferencia append, update y complete?
58. ¿Por qué exactly once debe analizarse extremo a extremo?
59. ¿Qué métricas detectan backpressure?
60. ¿Cuándo es contraproducente cachear?
61. ¿Qué diferencia RAG y fine-tuning?
62. ¿Cómo separar evaluación de recuperación y generación?
63. ¿Cuándo usar workflow?
64. ¿Cómo limitar Text-to-SQL?
65. ¿Qué hace trazable una afirmación?
66. ¿Qué riesgos introduce multimodalidad?
67. ¿Por qué eliminar nombres no anonimiza?
68. ¿Qué contiene una optimización reproducible?
69. ¿Cómo separar hecho, contexto e interpretación?
70. ¿Qué evidencia exigir antes de publicar?

9. Glosario

- **Backpressure:** entrada superior a capacidad.
- **Broker:** servidor Kafka.
- **Checkpoint:** estado persistido.
- **Consumer group:** consumidores que reparten particiones.
- **Embedding:** representación vectorial.
- **Event time:** momento real.
- **Idempotencia:** repetir no cambia el efecto.
- **Lag:** distancia entre offset disponible y procesado.
- **Microbatch:** pequeños lotes periódicos.
- **MMR:** equilibrio entre relevancia y diversidad.
- **Offset:** posición en partición.
- **Partición:** unidad ordenada y paralelizable.
- **RAG:** generación aumentada con recuperación.
- **Shuffle:** redistribución entre nodos.
- **Skew:** reparto descompensado.
- **Tool:** función externa de un agente.
- **Watermark:** umbral de datos tardíos.
- **Workflow:** secuencia explícita.

10. Bibliografía y documentación

- Apache Kafka Documentation. <https://kafka.apache.org/documentation/>
- Apache Spark Structured Streaming Guide. <https://spark.apache.org/docs/latest/streaming/index.html>
- Apache Spark SQL Performance Tuning. <https://spark.apache.org/docs/latest/sql-performance-tuning.html>
- Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly.
- Kreps, J. (2014). *I Heart Logs*. O'Reilly.
- Burns, B. (2018). *Designing Distributed Systems*. O'Reilly.
- Newman, S. (2021). *Building Microservices* (2nd ed.). O'Reilly.
- Lewis, P. et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *NeurIPS*.
- LangGraph Documentation. <https://docs.langchain.com/oss/python/langgraph/>
- NIST AI Risk Management Framework. <https://www.nist.gov/itl/ai-risk-management-framework>
- Reglamento (UE) 2016/679, Reglamento General de Protección de Datos.

Atribución recomendada

Fernández Isabel, A.; Madrueño Sierro, N.; Rodríguez Fernández, R. (2026). *Apuntes de Big Data Processing II*. Universidad Rey Juan Carlos. CC BY-SA 4.0.